Linked List Lab

In lecture we built `MyLinkedListDouble`: a list of `double`s, with `head == null` when empty. Most of its methods had special cases: the empty list, the first node, the last node.

In this lab you build a linked list of any type `E`, with one change: a **sentinel** node.

The lab has three parts:

1. the basic methods of a singly linked list with a sentinel;
2. more methods: search, iterator, `equals`, `middle`, `reverse`;
3. (optional) the same for a circular doubly linked list with a sentinel.

The sentinel

An extra node before the first one, holding no element (`data` is `null`). It is created with the list and never removed.

```
empty:     sentinel -> null                    tail = sentinel
[a, b]:    sentinel -> a -> b -> null          tail = b
```

Every node holding an element has a node before it. `tail` is never `null`.

- The sentinel is **not an element** of the list: it is not counted in `size`, has no position, and is never returned. A list whose only node is the sentinel is the empty list `[]`.
- It is a standard way to implement linked lists, not a workaround. It removes special cases from the code, and costs one extra node per list. No operation changes its complexity.

The files

`MyList.java`	the interface: what every method does. Do not modify
`SentinelLinkedList.java`	Parts 1 and 2
`SentinelDLinkedList.java`	Part 3
`Test.java`	the test program
`Driver.java`	try your list by hand, one command at a time

In the list classes, `toString` and `checkInvariants` are already written: do not modify them. `checkInvariants` walks the list and throws an exception if a link, `tail` or `size` is wrong. Everything compiles before you write anything.

Testing

```
javac Test.java
java Test
```

The tests are grouped by method, in the order you should write them. After every change to a list, `Test` calls `checkInvariants`: a wrong link or a forgotten `tail` fails at the operation that caused it, not later. When a group throws an exception, `Test` prints it with the line of your code that threw it and moves on to the next group.

To try your list by hand:

```
javac Driver.java
java Driver
```

Type `help` for the commands: `add x`, `add 2 x`, `remove 0`, `reverse`, ... After each command the driver prints the list and checks it with `checkInvariants`. Use it to reproduce a failing test step by step.

The elements in both programs are `String`s.

Part 1: the basics

Your TA will guide you through this part.

In `SentinelLinkedList.java`, write `nodeBefore` first: every method that works at a position uses it. Then the methods marked `TODO` in Part 1, in the order of the tests.

- Draw the list before and after each operation, for an empty list and for a list of one element.
- `add(e)`, `addFirst(e)` and `removeFirst()` are `O(1)`.
- Exceptions: as in lecture (`IndexOutOfBoundsException`, `NoSuchElementException`).

Part 2: more methods

In the same file, in this order:

- `indexOf`, `contains`: compare elements with `equals`, not `==`. You may assume no element is `null`.
- `Itr`, the iterator: `hasNext` and `next` first (`Test` uses them for `equals`), `remove` at the end.
- `equals(Object o)`: true if `o` is any `MyList` (singly or doubly linked) with equal elements in the same order. Walk both lists once, with their iterators. The type of `o` is `Object`: to call `MyList` methods on it, first check `o instanceof MyList`, then cast with `MyList<?> other = (MyList<?>) o;` (`?`: elements of unknown type).
- `middle()`: the element at position `size / 2`, in one pass and without `size`, `get` or `nodeBefore`. Hint: two references moving at different speeds.
- `reverse()`: changes the `next` references, creates no nodes. Hint: three references, `previous`, `current`, `following`. Then: where is `tail`?
- `Itr.remove()`: removes the element last returned by `next()`, in `O(1)`. Calling it before `next()`, or twice in a row, throws `IllegalStateException`.

Part 3 (optional): a circular doubly linked list

`SentinelDLinkedList.java`: a doubly linked list where the sentinel closes the circle.

```
empty:     sentinel.next = sentinel,   sentinel.prev = sentinel
[a, b]:    sentinel <-> a <-> b <-> (back to sentinel)
```

`sentinel.next` is the first node, `sentinel.prev` the last one: no `tail` field, and no `null` links.

Only the **representation** is circular, not the list. The list `[a, b]` still has a first element `a` and a last element `b`, and nothing comes after `b`: the sentinel marks where the list ends, as `null` did. Methods and iterators stop when they reach the sentinel, and never go around the circle.

Write the constructor first, then the private helpers `nodeAt`, `insertBefore` and `unlink`: most methods are one call to them. Then the rest, as for Part 1 and 2, plus `removeLast` and `descendingIterator`. For `middle`, try another way: two references walking from the two ends.

```
java Test dll
java Driver dll
```

What to submit

Upload the whole lab folder to Brightspace, with `SentinelLinkedList.java` completed. Include `SentinelDLinkedList.java` even if you did not do Part 3: `Test` and `Driver` use it, and do not compile without it.